

ESA — Universal Adaptive Runtime

Payment Infrastructure Implementation & Razorpay Buildathon PRD

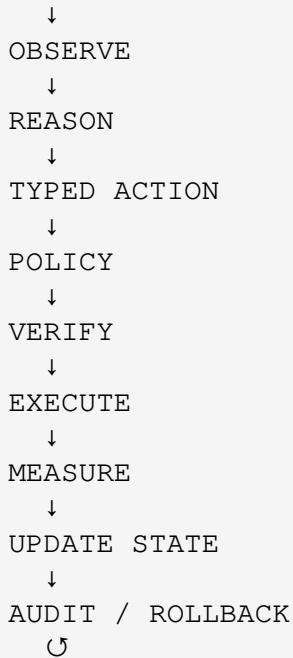
Project: Executable State Architecture (ESA) **Submission:** Razorpay Open Track **Primary Domain:** Payment infrastructure resilience
Architecture Type: Policy-bounded adaptive runtime **Implementation Target:** Working local prototype with reproducible end-to-end demo
Demo Freeze: 5 September 2026 **Document Status:** Final implementation PRD **Priority:** Working system > safety > measurable evidence > generality > feature breadth

1. EXECUTIVE DEFINITION

ESA is a policy-bounded adaptive runtime that converts changing workload conditions into **verified, typed, reversible infrastructure actions**.

The core architecture is:

```
INTENT
  ↓
CONSTRAINTS
  ↓
EXECUTABLE STATE
```



The central rule is:

AI may reason about infrastructure, but AI never owns infrastructure authority.

Agents may observe, diagnose, reason and propose.

Only deterministic runtime controls may:

- validate
 - authorize
 - modify
 - deny
 - execute
 - rollback
 - record the final decision
-

2. WHY ESA EXISTS

Traditional infrastructure automation is excellent at known conditions:

```
CPU > threshold  
→ scale out
```

but real distributed workloads often involve combinations of:

- regional traffic skew
- queue buildup
- latency degradation
- unhealthy nodes
- replica imbalance
- downstream pressure
- availability requirements
- cost constraints
- locality constraints
- consistency requirements

ESA explores whether bounded AI reasoning can help interpret those combinations while preserving deterministic infrastructure authority.

ESA is therefore NOT intended to claim invention of:

- autoscaling
- Kubernetes control loops
- self-healing
- observability
- infrastructure automation
- AIOps
- AI agents

The ESA contribution is the combination of:

```
state-aware reasoning  
+  
typed infrastructure actions
```

+
non-bypassable deterministic admission
+
state-version validity
+
measured action effects
+
replayable decision evidence
+
rollback

3. RAZORPAY POSITIONING

ESA MUST NOT claim knowledge of or access to Razorpay private infrastructure.

ESA MUST NOT claim:

- Razorpay lacks autonomous systems
- Razorpay lacks AI agents
- Razorpay lacks Kubernetes/cloud automation
- ESA replaces Razorpay infrastructure
- ESA is equivalent to Razorpay internal architecture

Razorpay already publicly operates in an AI-native and agentic direction, including agentic payments, agentic platform capabilities, AI-assisted payment workflows, and MCP/API-based agent interaction.

Therefore the ESA positioning is:

ESA is not another agentic payment experience. It is an adaptive infrastructure control-plane architecture designed to govern how AI-driven reasoning can safely influence distributed payment workloads.

The demo must create curiosity around:

"How could a payment platform let AI reason about infrastructure changes without giving AI unrestricted infrastructure authority?"

4. PRIMARY PRODUCT THESIS

ESA should demonstrate:

```
graph TD; A[Payment workload changes] --> B[Runtime state changes]; B --> C[AI reasons over bounded state]; C --> D[AI proposes typed action]; D --> E[Deterministic system validates action]; E --> F[State version is checked]; F --> G[Policy permits / modifies / denies]; G --> H[Single gateway executes]; H --> I[Actual runtime changes]; I --> J[Actual metrics change]; J --> K[Observed effect is compared with expected effect]; K --> L[New state is recorded]; L --> M[Decision can be replayed];
```

Payment workload changes
↓
Runtime state changes
↓
AI reasons over bounded state
↓
AI proposes typed action
↓
Deterministic system validates action
↓
State version is checked
↓
Policy permits / modifies / denies
↓
Single gateway executes
↓
Actual runtime changes
↓
Actual metrics change
↓
Observed effect is compared with expected effect
↓
New state is recorded
↓
Decision can be replayed

This is the minimum architecture necessary for ESA to be meaningful.

5. UNIVERSAL ARCHITECTURE

ESA MUST have a domain-independent core.

5.1 Generic core

The following must remain generic:

```
Intent
Constraint
ExecutableState
Observation
Diagnosis
Action
Policy
Verification
Execution
Effect
Rollback
Audit
Replay
```

5.2 Payment adapter

Only the following should be payment-specific:

```
PaymentEvent
payment.authorized
```

```
payment.captured  
payment.failed  
order.paid  
payment method class  
payment workload semantics  
payment correctness constraints  
webhook validation  
payment-specific safety rules
```

The architecture should make it possible to later connect:

```
Payment  
API Platform  
Database  
Streaming  
Logistics  
IoT  
Cloud Workload  
Data Pipeline  
Enterprise Service
```

without rebuilding the runtime.

6. CORE ENTITIES

6.1 Intent

Example:

```
{  
  "goal": "Keep P95 latency below 250 ms in IN-SOUTH",  
  "constraints": {
```

```
    "max_replicas": 8,  
    "allowed_regions": ["IN-SOUTH", "IN-WEST"],  
    "consistency": "strong",  
    "max_cost_increase_percent": 20  
  }  
}
```

6.2 Executable State

State must contain more than metrics.

Required:

```
workload_id  
domain  
version  
operational_state  
lifecycle_state  
topology  
region  
preferred_region  
fallback_regions  
workload_rate  
p50_latency  
p95_latency  
p99_latency  
error_rate  
queue_depth  
replica_count  
min_replicas  
max_replicas  
consistency_mode  
allowed_actions  
approval_requirement  
security_class  
rollback_reference
```

```
last_action
last_action_timestamp
```

State should therefore represent:

```
DATA
+
POLICY
+
CONSTRAINTS
+
TOPOLOGY
+
EXECUTION CONTEXT
+
VERSION
+
ROLLBACK CONTEXT
```

7. STATE VERSIONING — NON-NEGOTIABLE

Every state transition must produce a new version.

Example:

```
State = 481

Agent plans action against version 481

Another event arrives

State = 482
```

```
Old action reaches gateway
```

```
Gateway compares:
```

```
action.state_version = 481  
current.state_version = 482
```

```
→ REJECT STALE ACTION  
→ DO NOT EXECUTE  
→ RECORD REASON
```

This check **MUST** happen immediately before execution.

It **MUST NOT** depend on the LLM.

It **MUST NOT** exist only as a UI check.

The stale-state test is a mandatory live demo.

8. EVENT INGESTION

8.1 Synthetic generator

Synthetic workload generation is mandatory.

It must support:

```
steady  
burst  
regional skew  
latency increase  
failure increase
```

```
queue buildup  
node degradation  
mixed incident
```

The generator must produce deterministic seeds so experiments are reproducible.

8.2 Razorpay Test Mode

Razorpay Test Mode integration is preferred where practical.

The payment adapter may consume test events/webhooks, but the synthetic path must remain first-class.

The system must never depend entirely on external payment connectivity for the demo.

9. PAYMENT EVENT VALIDATION

Where webhook integration exists:

```
receive  
→ validate signature  
→ validate schema  
→ verify event ID  
→ deduplicate  
→ preserve ordering metadata  
→ normalize  
→ update workload state
```

Malformed and duplicate events must not trigger duplicate infrastructure

actions.

10. TELEMETRY

The state fabric must ingest:

```
throughput
P50 latency
P95 latency
P99 latency
failure ratio
queue depth
node health
replica count
replication lag
regional traffic
runtime capacity
action latency
```

Raw telemetry should be aggregated into bounded state summaries before model inference.

11. AGENT ARCHITECTURE

ESA uses four bounded cognitive roles.

11.1 Monitor Agent

Responsibilities:

- observe metrics
- detect abnormal conditions
- identify policy-relevant state changes

Restrictions:

```
READ ONLY
NO MUTATION
NO INFRASTRUCTURE CREDENTIALS
```

Example output:

```
{
  "condition": "REGIONAL_HOTSPOT",
  "region": "IN-SOUTH",
  "evidence": [
    "p95=642ms",
    "queue=1820",
    "regional_load=72%"
  ]
}
```

11.2 Diagnosis Agent

Inputs:

```
current state
recent state history
metrics
incident context
condition event
```

Output:

```
{
  "cause": "regional_capacity_pressure",
  "confidence": 0.91,
  "evidence_refs": [
    "metric:p95",
    "metric:queue",
    "metric:regional_load"
  ]
}
```

Diagnosis must reference actual evidence.

11.3 Planning Agent

The Planning Agent converts diagnosis + intent + current state into a typed action.

It MUST NOT return:

```
shell
kubectl
SQL
Python execution
arbitrary infrastructure commands
raw manifests
arbitrary tool instructions
```

It may return only registered action types.

11.4 Safety Agent

The Safety Agent reviews:

```
risk  
evidence  
constraints  
constitution  
action validity  
rollback availability
```

It may:

```
approve recommendation  
raise risk  
recommend modification  
recommend denial  
recommend human approval
```

It MUST NOT execute infrastructure.

12. TYPED ACTION INTERMEDIATE REPRESENTATION

ESA needs a stable domain-independent Action IR.

Required fields:

```
{  
  "action": "CREATE_REPLICA",  
  "request_id": "req_123",  
  "workload_id": "w_1042",  
  "state_version": 481,  
  "parameters": {  
    "target_region": "IN-SOUTH",
```

```
    "replica_delta": 1
  },
  "reason": "P95 latency above policy threshold",
  "evidence_refs": [
    "metric:p95",
    "metric:queue"
  ],
  "confidence": 0.91,
  "risk": "LOW",
  "expected_effect": {
    "p95_latency_delta_ms": -80,
    "queue_delta": -500
  },
  "rollback": {
    "enabled": true
  }
}
```

This Action IR is one of the most important pieces of the universal ESA architecture.

13. SUPPORTED ACTIONS

Mandatory:

```
CREATE_REPLICA
SHIFT_ROUTE
ROLLBACK
```

Optional only after the mandatory path works:

```
REMOVE_REPLICA
MIGRATE_PARTITION
THROTTLE_WORKLOAD
```

Do not add additional actions merely to make the architecture look larger.

14. ACTION SCHEMA VALIDATION

Every action **MUST** be validated before policy evaluation.

Reject:

```
unknown action
missing required fields
invalid parameters
wrong types
out-of-range values
invalid region
invalid state version
missing rollback data when required
```

Schema validation **MUST** be deterministic.

15. ESA CONSTITUTION

Priority order:

1. Safety
2. Security / Compliance
3. Payment Correctness
4. Availability

5. Reversibility
6. Cost
7. Performance
8. Operator Intent

Lower priorities MUST NEVER override higher priorities.

Example:

```
Cost optimization  
CANNOT  
override  
payment correctness
```

16. POLICY ENGINE

The Policy Engine is deterministic.

Possible outcomes:

```
ALLOW  
MODIFY  
DENY  
REQUIRE_APPROVAL  
STALE_STATE
```

Example rules:

```
IF action not registered  
    DENY  
  
IF state_version != current_state_version  
    STALE_STATE
```

```
IF target_region not allowed
    DENY
```

```
IF replica_count + requested_delta > max_replicas
    DENY
```

```
IF risk == HIGH AND approval_required
    REQUIRE_APPROVAL
```

```
IF rollback_required AND rollback unavailable
    DENY
```

The evaluator **MUST** demonstrate:

```
1 ALLOW
1 DENY
1 STALE_STATE
```

live.

17. DECISION VERIFIER

The Decision Verifier performs the final pre-execution check.

It **MUST** read CURRENT state.

It must verify:

```
state version
action schema
current metrics
current topology
permissions
replica bounds
```

```
region limits
risk level
approval requirement
rollback availability
policy result
```

The Planning Agent's snapshot must never be trusted as current truth.

18. ACTION GATEWAY — PRIMARY DIFFERENTIATOR

The Action Gateway is the **ONLY** path capable of requesting runtime mutation.

Allowed:

```
Agent
↓
Action IR
↓
Policy
↓
Verifier
↓
Gateway
↓
Executor
```

Forbidden:

```
Agent → shell
Agent → kubectl
Agent → database
```

```
Agent → Docker  
Agent → Kubernetes  
Agent → cloud control plane
```

All runtime credentials, where needed, belong only to the controlled execution boundary.

The gateway must produce a signed or integrity-protected action envelope containing:

```
request_id  
action  
state_version  
policy_version  
decision_id  
parameters  
rollback_reference  
timestamp
```

19. NON-BYPASSABILITY REQUIREMENT

The evaluator must be able to prove:

```
No agent service  
can directly mutate runtime state.
```

Minimum verification:

- agent containers have no runtime admin credentials
- executor credentials are isolated
- gateway is required by architecture
- executor rejects requests without valid gateway envelope

- direct malformed execution attempt is blocked
 - audit shows gateway decision before execution
-

20. RUNTIME EXECUTOR

Preferred demo target:

```
Kubernetes kind/minikube
```

Equivalent controlled multi-service runtime is acceptable.

The executor must perform REAL mutations.

Example:

```
workers = 3
      ↓
CREATE_REPLICA
      ↓
workers = 4
```

or:

```
IN-SOUTH = 80%
IN-WEST = 20%

      ↓

SHIFT_ROUTE

IN-SOUTH = 50%
IN-WEST = 50%
```

A dashboard animation is NOT runtime execution.

21. PRIMARY DEMO

Regional Payment Workload Hotspot

Stage A — Healthy

```
traffic stable  
p95 stable  
queue stable  
replicas stable
```

Stage B — Incident

Inject:

```
3x-5x workload burst
```

or:

```
regional traffic skew
```

Stage C — Detection

Expected:

```
P95 ↑  
queue ↑  
regional imbalance ↑
```

Stage D — Monitor

Detect:

```
REGIONAL_HOTSPOT
```

Stage E — Diagnosis

Explain the likely cause using actual evidence.

Stage F — Planning

Generate:

```
CREATE_REPLICA
```

or:

```
SHIFT_ROUTE
```

Stage G — Safety

Review risk and policy.

Stage H — Verification

Check current state/version.

Stage I — Gateway

Permit or reject.

Stage J — Runtime

Perform the actual mutation.

Stage K — Measurement

Collect actual post-action metrics.

Stage L — Effect Verification

Compare:

```
expected_effect  
vs  
observed_effect
```

Example:

```
Expected P95 improvement: -80ms  
Observed P95 improvement: -17ms
```

ESA must not blindly assume that execution equals success.

Stage M — State Update

Record observed effect in new state.

Stage N — Audit

Show full lineage.

22. EFFECT VERIFICATION

Every mutation should have:

```
expected_effect
observed_effect
effectiveness
```

Example:

```
{
  "expected": {
    "p95_delta_ms": -80
  },
  "observed": {
    "p95_delta_ms": -61
  },
  "effectiveness": 0.76
}
```

Minimum implementation:

```
expected metric target
observed metric result
difference
success / underperformance / failure
```

This is a major ESA differentiator.

23. CLOSED-LOOP ADAPTATION

Execution is NOT the end.

After the action:

```
MEASURE
↓
COMPARE EXPECTED VS OBSERVED
↓
UPDATE STATE
↓
CHECK OBJECTIVE
↓
DECIDE WHETHER FURTHER ADAPTATION IS REQUIRED
```

Possible results:

```
OBJECTIVE_MET
PARTIALLY_MET
UNDERPERFORMED
FAILED
```

For the demo, UNDERPERFORMED may trigger replanning or safe no-op.

24. ROLLBACK

Rollback must use actual stored rollback material.

Mandatory failure demo:

```
approved action
↓
runtime mutation begins
↓
failure injected
```

```
↓  
executor detects failure  
↓  
rollback / compensating action  
↓  
previous valid state restored
```

Audit:

```
original action  
failure  
rollback action  
restored state  
final outcome
```

25. SAFE MODEL FAILURE

If:

```
LLM timeout  
model unavailable  
invalid model output  
malformed action  
agent service unavailable
```

the system must NOT execute unsafe changes.

Preferred behavior:

```
safe no-op  
OR  
deterministic rule fallback  
OR  
human approval
```

OR
replan

The system must fail closed.

26. HUMAN APPROVAL

High-risk actions must support:

REQUIRE_APPROVAL

Approval record:

```
decision_id  
action  
risk  
reason  
evidence  
expected_effect  
rollback_plan  
approver  
timestamp
```

The executor must reject high-risk actions that do not satisfy the required approval condition.

27. AUDIT MODEL

Every decision must have a unique correlation ID.

Required audit fields:

```
event_id
trace_id
decision_id
workload_id
state_version
policy_version
agent outputs
evidence references
proposed action
policy result
verification result
final action
before_state
after_state
observed_effect
execution outcome
rollback status
timestamp
```

Audit records should be append-only for the demo.

28. DECISION REPLAY

A previous action must be replayable from stored artifacts without requiring a new model generation.

Replay input:

```
state summary
state version
intent/constraints
policy version
```

```
action artifact
evidence references
```

Replay must be able to reconstruct:

```
policy decision
verification result
action validity
```

The original LLM call is NOT required for deterministic replay.

29. OBSERVABILITY

Dashboard sections:

Payment / Workload Health

```
throughput
P50
P95
P99
failure ratio
queue
```

Runtime Topology

```
nodes
replicas
regional routing
```

workers
capacity

Agent Reasoning

condition
evidence
diagnosis
confidence
proposal

Governance

allowed
modified
denied
stale
approval-required

Execution

before
action
after
duration
observed effect
rollback

AI Cost

```
model  
calls  
latency  
tokens if available  
cache hit rate
```

30. BENCHMARKING

Compare:

```
BASELINE A:  
Rule-only orchestration
```

```
BASELINE B:  
ESA agent-assisted orchestration
```

Same workload seeds must be used.

Required scenarios:

```
steady  
3x burst  
regional skew  
node failure  
queue buildup  
mixed incident
```

Minimum metrics:

```
P95 latency  
hotspot resolution time  
recovery time  
queue drain time
```

```
action latency
policy safety rate
policy violation rate
rollback success
replica efficiency
AI inference latency
AI token cost
```

31. BENCHMARK QUALITY

Benchmark runs must be:

```
repeatable
seeded
recorded
comparable
```

Do not use:

```
fabricated improvements
single cherry-picked runs
different workloads for baseline and ESA
UI-generated numbers
```

Store raw benchmark traces.

Report:

```
mean
median where useful
variance or spread where practical
best/worst where useful
```

For the buildathon, the goal is not statistically perfect research.
The goal is credible engineering evidence.

32. SAFETY TEST SUITE

The project **MUST** automatically test:

Test 1 — Unknown Action

EXECUTE_SHELL

Expected:

DENY

Test 2 — Out-of-Bounds Replicas

CREATE_REPLICA +5000

Expected:

DENY

Test 3 — Unauthorized Region

```
SHIFT_ROUTE → invalid region
```

Expected:

```
DENY
```

Test 4 — Stale State

```
action.version = 481  
current.version = 482
```

Expected:

```
STALE_STATE
```

Test 5 — Missing Approval

High-risk action without required approval.

Expected:

```
REQUIRE_APPROVAL / DENY
```

Test 6 — Invalid Model Output

Expected:

```
NO EXECUTION
```

Test 7 — Agent Failure

Expected:

```
safe operation
```

Test 8 — Runtime Failure

Expected:

```
rollback / compensation
```

33. PAYMENT DATA SAFETY

Never expose to the model:

```
card numbers  
CVV  
API keys  
authentication tokens  
payment secrets  
unnecessary PII
```

Use:

```
pseudonymous identifiers  
aggregated metrics  
payment method class  
region  
workload metadata
```

latency buckets
failure ratios

The AI sees workload state, not sensitive payment credentials.

34. FAILURE HANDLING

Failure	Required response
Duplicate event	ignore duplicate execution, preserve audit
Out-of-order event	reconcile/defer
Agent unavailable	safe fallback
LLM timeout	no unsafe mutation
Invalid action	reject
Stale state	reject and replan
Policy violation	deny and record
Runtime failure	rollback/compensation
Missing approval	block
Gateway unavailable	do not execute

35. GENERALIZED ESA API MODEL

The domain-independent runtime should conceptually expose:

```
POST /intent
POST /state
POST /observe
POST /actions/validate
POST /policy/evaluate
POST /verify
POST /execute
POST /rollback
GET /decisions/{id}
GET /decisions/{id}/replay
GET /state/{id}
```

Payment adapters should sit above or beside the generic runtime.

36. REPOSITORY ARCHITECTURE

Recommended:

```
/apps
  /event-gateway
  /runtime-api
  /agent-service
  /policy-service
  /action-gateway
  /runtime-executor
  /dashboard

/core
  /state
  /intent
  /constraints
  /actions
```

```
/policy
/verification
/effects
/rollback
/audit
/replay

/agents
  /monitor
  /diagnosis
  /planning
  /safety

/adapters
  /payment
  /synthetic

/runtime
  /kubernetes
  /simulator
  /failure-injection

/observability
  /metrics
  /traces

/benchmarks
  /workloads
  /baselines
  /results

/tests
  /unit
  /integration
  /security
  /e2e
  /benchmark
```

The exact folder structure may differ, but the logical separation **MUST**

exist.

37. TECHNOLOGY REQUIREMENTS

Preferred prototype stack:

Runtime:

Python/FastAPI acceptable for prototype

Agent layer:

local SLM through Ollama or equivalent

State:

PostgreSQL + Redis

Schema:

Pydantic / JSON Schema

Messaging:

NATS optional

Runtime:

kind/minikube/Kubernetes or controlled equivalent

Observability:

OpenTelemetry

Prometheus

Grafana

Containers:

Docker

CI:

Do NOT add:

```
WASM runtime
Raft cluster
custom database
multi-cluster orchestration
predictive ML
```

unless the mandatory demo already works.

38. MVP BOUNDARY

Mandatory:

```
synthetic workload
state fabric
4 bounded agents
typed Action IR
Policy Engine
Decision Verifier
Action Gateway
real runtime mutation
CREATE_REPLICA
SHIFT_ROUTE
stale-state rejection
allow/deny policy cases
observability
audit
rollback
rule-only benchmark
```

Optional:

third action
Razorpay Test Mode webhook integration
decision caching
token optimization
rich replay visualization
multi-node message bus

Deferred:

WASM entities
advanced consensus
global placement optimization
predictive scaling
full distributed state engine
production secrets platform

39. PRIMARY IMPLEMENTATION PRIORITIES

Implementation order MUST be:

1. Runtime scaffold
2. State model
3. State versioning
4. Action IR
5. Policy Engine
6. Decision Verifier
7. Action Gateway
8. Real runtime mutation
9. Synthetic workload
10. Monitor
11. Diagnosis

12. Planning
13. Safety
14. Effect measurement
15. Audit
16. Rollback
17. Benchmark
18. Razorpay Test Mode integration if stable
19. Presentation polish

Do not reverse this order for UI reasons.

40. DEFINITION OF DONE

ESA is NOT complete because:

```
UI works
agents return text
README is complete
services start
architecture diagram looks correct
```

ESA is complete only if this loop works repeatedly:

```
WORKLOAD
↓
STATE UPDATE
↓
INCIDENT DETECTION
↓
DIAGNOSIS
↓
TYPED ACTION
↓
SAFETY
```

↓
DETERMINISTIC POLICY
↓
STATE VERSION CHECK
↓
ACTION GATEWAY
↓
REAL MUTATION
↓
OBSERVED EFFECT
↓
STATE UPDATE
↓
AUDIT

and:

unsafe action → blocked
stale action → rejected
runtime failure → rollback
model failure → safe fallback

41. MANDATORY END-TO-END ACCEPTANCE TESTS

AC-01

A workload event changes observable runtime state.

AC-02

At least one hotspot/burst is detected automatically.

AC-03

Monitor produces evidence.

AC-04

Diagnosis references live evidence.

AC-05

Planning produces valid Action IR.

AC-06

Safety produces a risk/policy assessment.

AC-07

Policy Engine can ALLOW a valid action.

AC-08

Policy Engine can DENY an invalid action.

AC-09

Gateway rejects a stale state version.

AC-10

Agents cannot execute arbitrary commands.

AC-11

CREATE_REPLICA executes and changes actual runtime state.

AC-12

SHIFT_ROUTE executes and changes actual routing state.

AC-13

Metrics respond to the mutation.

AC-14

Expected vs observed effect is recorded.

AC-15

Audit lineage reconstructs the decision.

AC-16

Decision replay works without new LLM generation.

AC-17

Injected runtime failure triggers rollback.

AC-18

LLM timeout cannot cause unsafe mutation.

AC-19

No sensitive payment data reaches the AI layer.

AC-20

Rule-only and ESA benchmark results are reproducible.

AC-21

Fresh setup can run the complete demo.

AC-22

Five-minute demo runs without source-code edits.

42. ANTI-MOCK ACCEPTANCE

For EVERY major feature the implementation must answer:

```
Where implemented?  
Which service?  
Which function/API?  
What input triggers it?  
What state does it modify?  
What test proves it?  
What runtime evidence proves it?
```

Any feature without evidence is:

43. FINAL DEMO FLOW

Recommended 5-minute flow:

00:00

Problem:

Payment infrastructure changes faster than static rules.

00:30

Healthy runtime.

01:00

Trigger regional hotspot.

01:20

P95 + queue degradation appears.

01:40

Monitor detects condition.

02:00

Diagnosis explains cause.

02:20

Planner emits typed action.

02:40

Safety + Policy + Verifier.

03:00

Gateway permits action.

03:20

Actual runtime mutation.

03:40

Metrics improve.

04:00

Show expected vs observed effect.

04:15

Show audit/replay lineage.

04:30

Inject execution failure.

04:45

Rollback.

05:00

Show rule-only vs ESA benchmark.

44. RAZORPAY CURIOSITY MOMENT

The demo must deliberately expose one moment that ordinary AIOps systems do not communicate clearly:

Agent says:

"I want to create one replica."

Gateway says:

"Rejected.

The agent planned against state version 481.

```
Current state is 482."
```

Then show:

```
No infrastructure mutation occurred.  
Decision recorded.  
Agent must replan.
```

This demonstrates:

```
AI reasoning  
≠  
infrastructure authority
```

That is more valuable than showing another successful autoscaling animation.

45. SECOND CURIOSITY MOMENT

After successful execution:

```
AI expected:  
P95 -80ms  
  
Runtime observed:  
P95 -23ms
```

ESA records:

```
UNDERPERFORMED
```

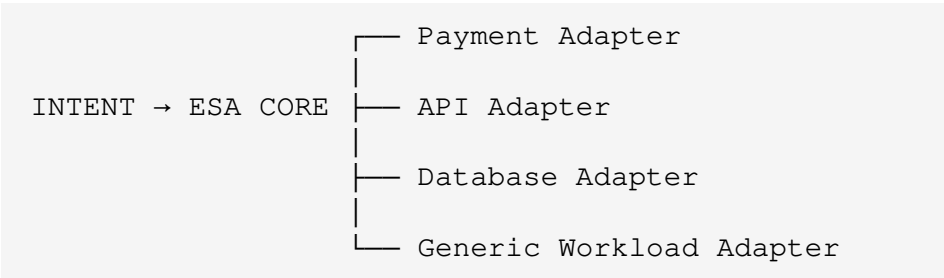
and updates state.

This demonstrates that ESA does not blindly trust the model or the success of the command.

46. UNIVERSALITY PROOF

The payment demo must be implemented through adapters.

Conceptually:



A minimal non-payment example SHOULD be possible after the payment demo by replacing the event adapter and workload schema without changing:

- Policy Engine
- Action IR
- Verifier
- Gateway
- Audit
- Rollback
- Effect measurement

This is the strongest evidence that ESA is a runtime architecture rather than a payment-specific automation script.

47. WHAT ESA SHOULD CLAIM

Preferred:

ESA explores a policy-bounded adaptive runtime where probabilistic agents reason over executable workload state, but deterministic controls retain final authority over infrastructure changes.

Preferred:

ESA converts operational intent and live workload state into typed, verifiable and reversible runtime actions.

Preferred:

ESA treats post-action measurement and decision lineage as first-class parts of autonomous infrastructure control.

48. WHAT ESA MUST NOT CLAIM

Do not claim:

"We invented autonomous infrastructure."

"Razorpay does not have autonomous infrastructure."

"No existing system does this."

"ESA can safely control production automatically."

"AI decisions are always correct."

"ESA provides zero downtime."

"ESA provides perfect security."

"ESA controls Razorpay production."

"ESA reproduces Razorpay internal architecture."

49. SUCCESS METRICS

Minimum Buildathon evidence:

- 1 hotspot scenario
- 2 working typed actions
- 1 allowed policy decision
- 1 blocked policy decision
- 1 stale-state rejection
- 1 real topology mutation
- 1 effect measurement
- 1 rollback
- 1 replayable audit
- 1 rule-only comparison
- 1 reproducible clean run

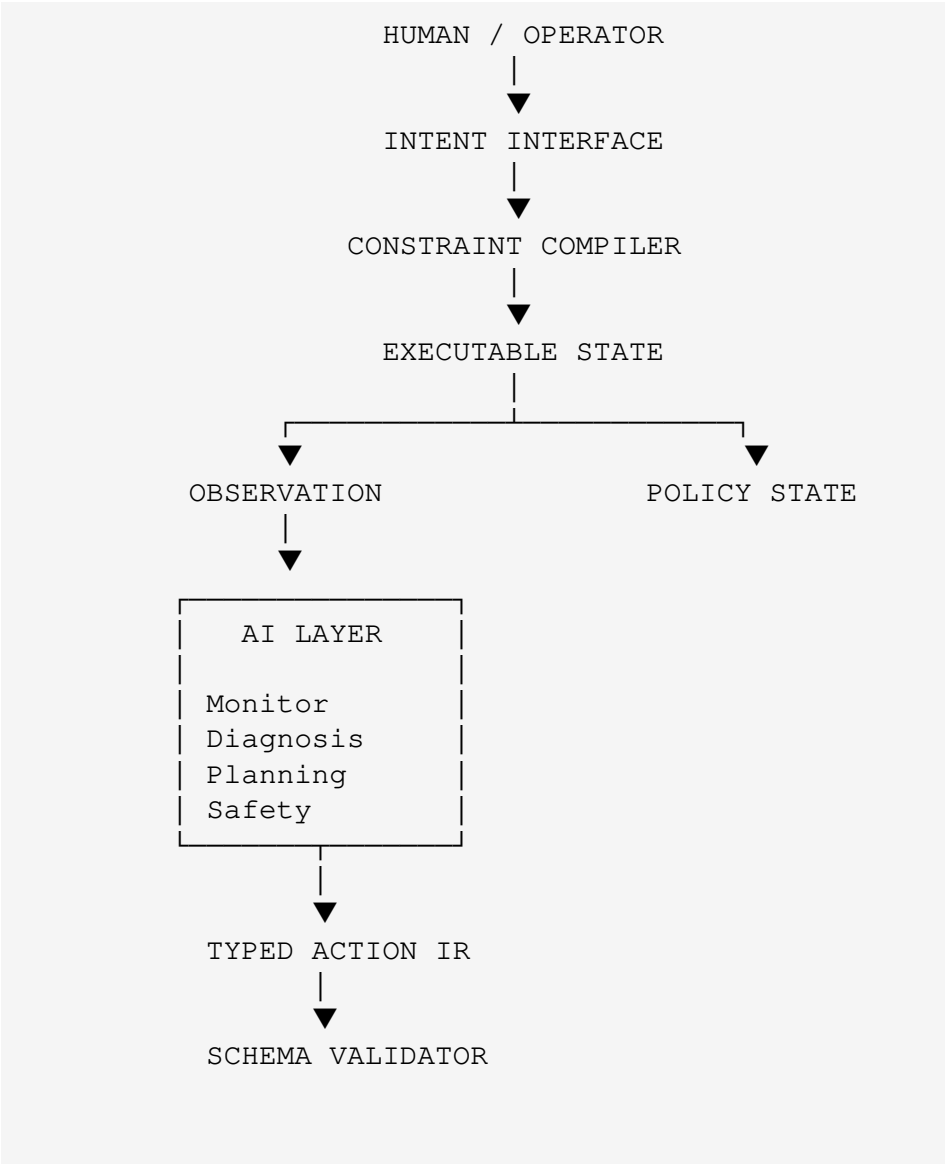
Strong-result targets:

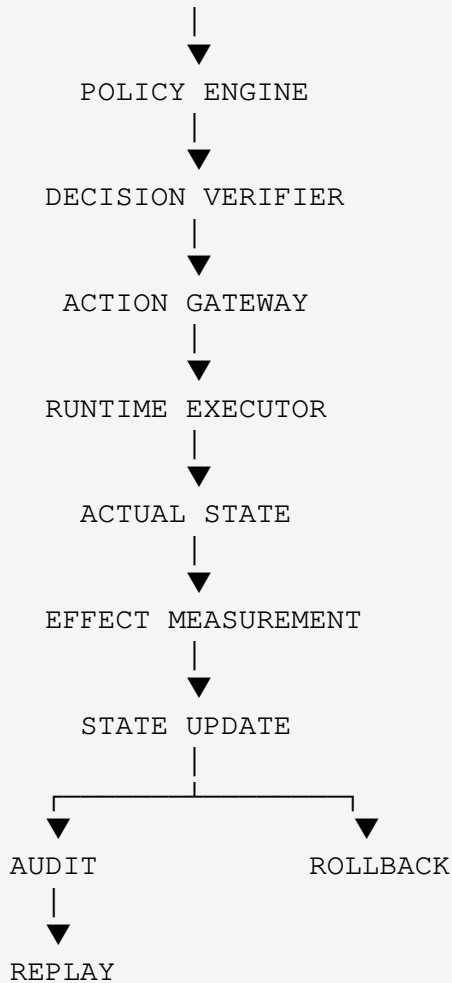
- lower hotspot resolution time
- lower P95 latency after intervention
- reduced queue drain time
- higher action safety
- successful rollback
- zero arbitrary agent execution
- reproducible results

No fixed percentage improvement should be fabricated in advance.

50. FINAL ARCHITECTURAL FORM

ESA should converge to:





51. FINAL PRIORITY RULE

When implementation time is limited:

1. End-to-end reliability

2. Non-bypassable safety
3. Real runtime mutation
4. Stale-state protection
5. Rollback
6. Measured effect
7. Audit/replay
8. Benchmark
9. Universal core separation
10. Razorpay adapter
11. UI polish
12. Stretch research

Never reverse this order.

52. FINAL DEFINITION OF ESA

ESA is a universal adaptive runtime architecture in which intent and live executable state guide bounded AI reasoning, typed actions are admitted through deterministic policy and state-version verification, infrastructure changes occur only through a controlled gateway, and every action is measured, auditable, replayable, and reversible.

The payment implementation is the first concrete operating domain.

The long-term architecture is the runtime itself.

53. FINAL RELEASE GATE

The project may be declared:

DEMO READY

only when:

```
[PASS] workload generation
[PASS] state update
[PASS] incident detection
[PASS] diagnosis
[PASS] typed planning
[PASS] safety review
[PASS] deterministic policy
[PASS] stale-state rejection
[PASS] gateway-only execution
[PASS] real runtime mutation
[PASS] observed effect
[PASS] audit
[PASS] replay
[PASS] rollback
[PASS] rule-only benchmark
[PASS] reproducible setup
```

If any P0 item fails:

```
STATUS = NOT DEMO READY
```

P0 items:

```
state versioning
typed action validation
policy enforcement
decision verification
action gateway
runtime mutation
rollback
agent execution isolation
```

54. FINAL IMPLEMENTATION REPORT

The coding agent must finish with:

```
ESA FINAL VERIFICATION
```

```
Demo Readiness: XX%
```

```
PRD Compliance: XX%
```

```
Safety Compliance: XX%
```

```
Universal Runtime Maturity: XX%
```

```
Research Differentiation: XX%
```

```
P0:
```

```
PASS / FAIL
```

```
P1:
```

```
PASS / FAIL
```

```
Real Runtime Mutation:
```

```
PASS / FAIL
```

```
Stale-State Rejection:
```

```
PASS / FAIL
```

```
Gateway-Only Authority:
```

```
PASS / FAIL
```

```
Rollback:
```

```
PASS / FAIL
```

```
Expected vs Observed Effect:
```

```
PASS / FAIL
```

```
Replay:
```

```
PASS / FAIL
```

Benchmark:
PASS / FAIL

Razorpay Adapter:
PASS / FAIL

Final Status:
NOT READY
PARTIAL
DEMO READY
DEMO VERIFIED
RESEARCH DIFFERENTIATION VERIFIED

Critical Remaining Gaps:
1.
2.
3.

Evidence:
1.
2.
3.

55. FINAL RULE

Do not optimize ESA to look autonomous.

Optimize ESA to prove that autonomy is **bounded, state-aware, deterministic at the authority boundary, measurable, reversible, and generalizable**.

The final demo must make the following visible:

THE MODEL CAN THINK.

THE MODEL CANNOT COMMAND.
THE RUNTIME CAN VERIFY.
THE RUNTIME CAN EXECUTE.
THE RUNTIME CAN MEASURE.
THE RUNTIME CAN REJECT.
THE RUNTIME CAN ROLLBACK.
THE DECISION CAN BE REPLAYED.

That is the ESA implementation target.